SN1 Solar Node

Granz Scientific LLC

June 8, 2026
Document Version 0.3

Contents

1 Introduction

The SN1 Solar Node from Granz Scientific LLC is intended to provide a straightforward development platform for solar-powered IoT devices. It provides a solar battery charger for one or two 18650-sized lithium cells, battery protection circuitry, an ultra-efficient 3.3V power supply, and an ESP32-C3 controller module with USB port for easy development without additional specialized hardware. All of this is housed in an IP65-rated enclosure with an integrated 0.8-watt solar panel. If ESPHome firmware is installed on the device, it may be integrated quickly with Home Assistant running on the local network.

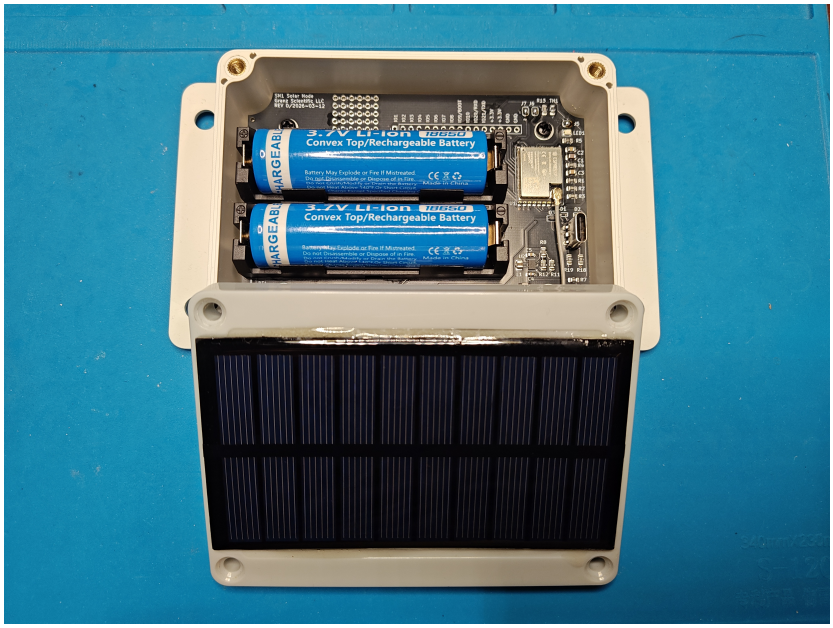

Figure 1: SN1 Solar Node

Upon integration into Home Assistant, the SN1 Solar Node will show as in Figure 3. Additional sensors will show up based on the user's additional hardware and firmware configuration.

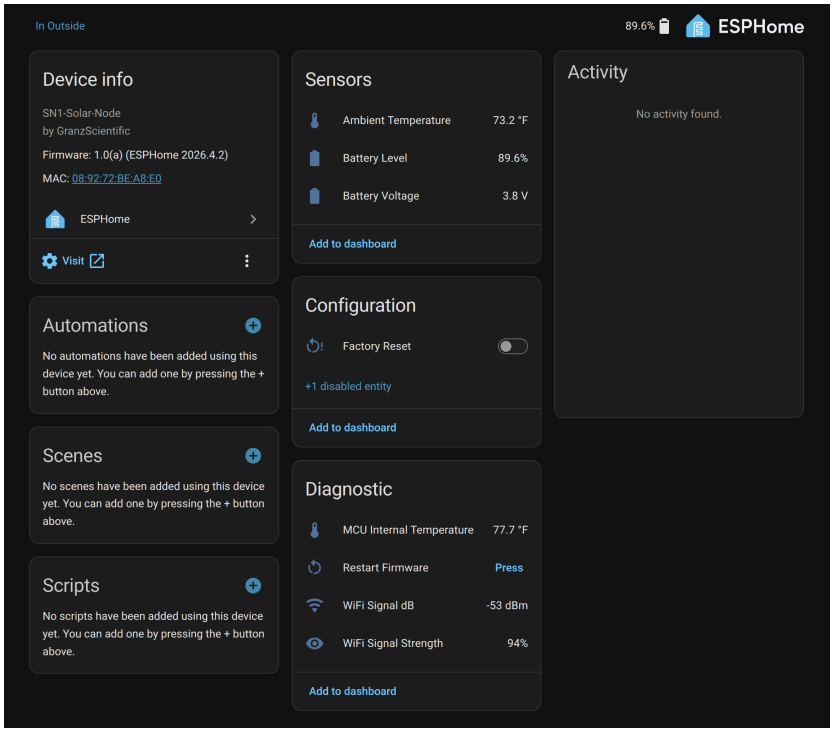

Figure 2: Home Assistant Device Page

2 Specifications

Specification	Value	Units
Controller	ESP32-C3	N/A
System Voltage	3.3	volts
Solar Panel Voltage (Full-sun)	5	volts
Solar Panel Current (Full-sun)	160	mAmperes
Solar Panel Wattage (Full-sun)	0.8	watts
Switched Off Discharge Current	3.2	uAmperes
Switched On WiFi AP Mode Discharge Current	85-100	mAmperes
Switched On WiFi Connected Mode Discharge Current	20-50	mAmperes
Switched On Deep Sleep Discharge Current	40	uAmperes
Max Available Supply Current (For User-added Parts)	200	mAmperes
Operating Temperature	-20 to +85	˚C
WiFi Working Range	150*	meters
Dimensions (Including Mounting Tabs)	150*85*35	millimeters
Dimensions (Not Including Mounting Tabs)	115*85*35	millimeters
Enclosure Rating	IP65	N/A

*Dependent on installed location, environment, and WiFi router

Table 1: Device specifications.

3 System Layout

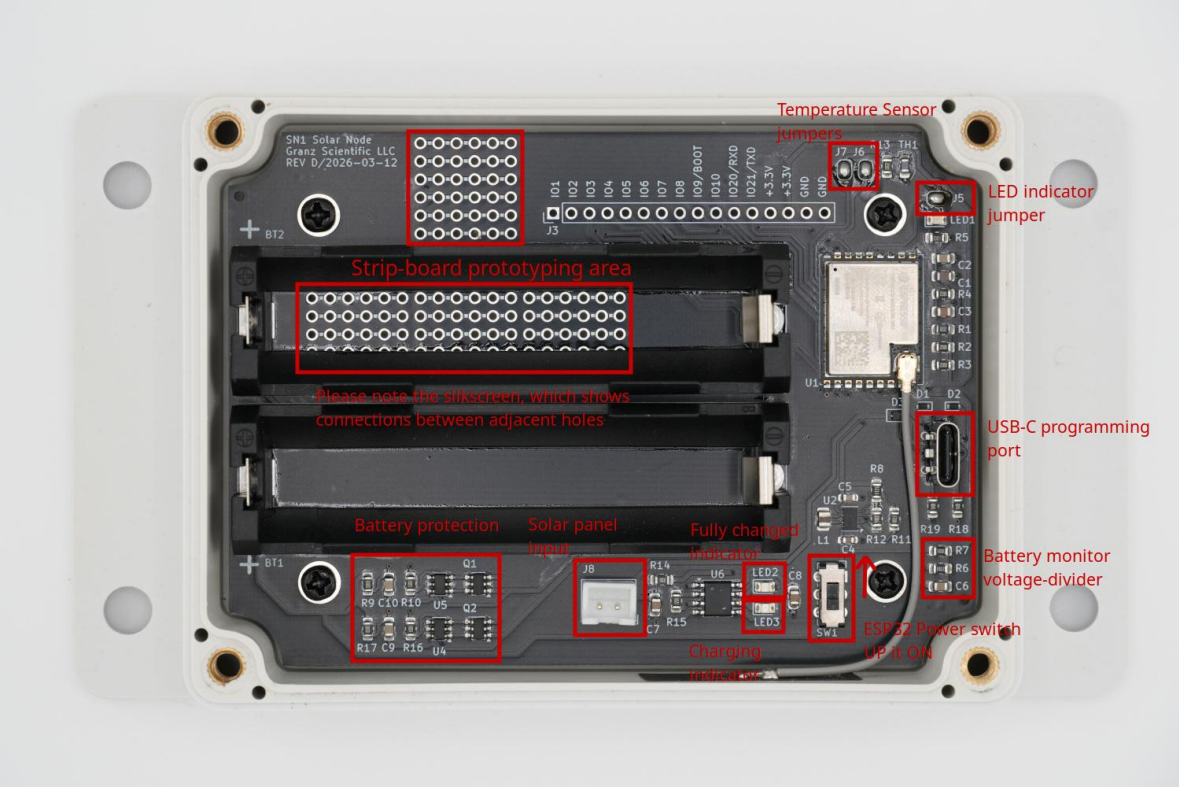

Figure 3: System Layout

4 Hardware Setup

Remove the four screws from the top of the device to open the lid for initial hardware configuration.

The device includes and onboard LED and temperature sensor which may be enabled by soldering jumpers on the PCB. Referring to the system layout, there is a single jumper that should be solder-bridged to connect the LED to IO6 of the ESP32-C3 module, and two jumpers that must be solder-bridged to connect the temperature sensor to IO3 and IO1 of the ESP32-C3 module. IO1 is used to power the themistor circuit and IO3 is used to read the voltage across the thermistor. Keep in mind that if any of these features are enabled via the solder jumpers that these IO pins will not be available for other user functions.

Install one or two 18650 rechargeable cells (depending on SN1 version) into internall battery holders and turn on the power switch. The device ships with a basic ESPHome firmware image that allows the device to be connected to a WiFi network and be added to Home Assistant.

WARNING: Once power is applied (switched on) you will have 5 minutes to configure the device before it automatically enters sleep mode.

5 Firmware Setup

NOTE: The following information assumes that the factory ESPHome firmware is installed.

Using a WiFi enabled device (phone, tablet, or computer) connect to the device's access point, which will show up with a name such as "**GS SN1 Solar Node f6ddb0**". The last 6 characters of the name represent the last 6 hexadecimal digits of the MAC and will vary between individual devices. The default device WiFi access point password is "**12345678**".

After connecting to the device's WiFi access point, opening a web browser should automatically take you to a setup page where you can enter your local network WiFi credentials. After clicking Save, the device will reboot and attempt to connect to your local network WiFi.

You will now have 5 minutes to complete the setup with the Home Assistant connection before the device goes to sleep to prevent draining the battery.

In Home Assistant, go to Settings → Devices & services and look for a pop-up asking if you want to add the new device. After clicking to add the device, it will be added to your Home Assistant ESPHome devices and ready for use. Keep in mind that the device will take a few samples at this point, and then immediately go to sleep for 15 minutes.

6 Customization & Over-The-Air Updates

By default, the device does not use encryption for the API connection or have an OTA update password. **It is recommended that all users import the device into their enviroment (ESPHome-Builder or ESPHome CLI) and update the device to use encryption for security purposes.**

First, you will need to add a flag to Home Assistant which the device will pickup and which will prevent it from going to sleep every 15 minutes. This is important to be able to update the firmware wirelessly, since when the device is sleeping between samples it will be offline.

WARNING: When the flag is set to "on" in Home Assistant, the device (and other GS devices) will not go to sleep, potentially draining the battery faster than normal while left in this state. After you are finished with your updates, be sure to set the flag back to "off".

Add the following to your Home Assistant configuration.yaml:

```
input_boolean:
  ota_update_mode:
    name: "OTA Update Mode"
    initial: off
    icon: mdi:chip
```

Figure 4: OTA Update Mode Switch

You will need to restart Home Assistant afterwards so that it will reload the configuration.yaml file. **After Home Assistant has restarted, add the entity "OTA Update Mode" to a dashboard and turn the switch to "on".** It will show up as a switch as shown in Figure 4. Please note that if the device is currently sleeping, it may take up to 15 minutes before the device is online and stays awake, depending on when the last sample was taken and where it is in it's sleep cycle.

There are two options for customizing the ESPHome firmware:

- Install the ESPHome Device Builder add-on in Home Assistant (recommended)
  The device should automatically show up in ESPHomeBuilder and give you the option to "Take Control". This will generate a YAML file for you which references the github repository.
- Install ESPHome CLI
  You'll need to clone the project git repository and then compile and upload the firmware manually.

To customize the devices ESPHome firmware, first put the device into OTA Update Mode using the switch added to Home Assistant as described above, which will prevent the device for going into sleep mode.

Open the ESPHomeBuilder add-on in Home Assistant, and watch for the device to appear after it has come online (max 15 minutes). Click the "Take Control" button and follow the prompts to install a custom built ESPHome image with your additional configuration. The factory configuration will be automatically included and merged with any additional sections created for your custom application.

To enable the onboard LED add this your custom YAML configuration file:

```
output:
  - platform: ledc
    id: led_out
    pin: GPIO6
    channel: 4
```

To enable the onboard temperature sensor add this your custom YAML configuration file:

```
switch:
  - platform: gpio
    id: ntc_pwr
    pin: GPIO3
    internal: True # Hide from home assistant

sensor:
  - platform: ntc
    name: "Ambient Temperature"
    sensor: ntc_resistance_sensor
    calibration:
      b_constant: 3950
      reference_temperature: 25°C
      reference_resistance: 10kOhm
  - platform: resistance
    id: ntc_resistance_sensor
    name: "NTC Resistance"
    sensor: ntc_source_sensor
    configuration: DOWNSTREAM
    resistor: 10.0kOhm
    internal: True # Hide from home assisstant
  - platform: adc
    id: ntc_source_sensor
    name: "NTC Voltage"
    pin: GPIO1
    # Never update, trigger it from interval block
    update_interval: never
    attenuation: 12dB
    internal: True # Hide from home assisstant

interval:
  - interval: 5s
    startup_delay: 5s
    then:
      - switch.turn_on: ntc_pwr
      - component.update: ntc_source_sensor
      - switch.turn_off: ntc_pwr
```

After you have finished updating the firmware using the ESPHomeBuilder add-in, make sure to set the OTA Update Mode switch back to “off” to avoid draining the SN1’s battery.